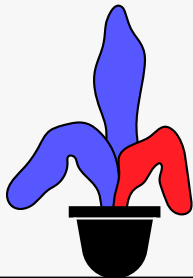


Elementos básicos de JAVA

Introducción a la Programación
Orientada a Objetos





JAVA

- Lenguaje de programación presentado en 1995
 - Soporta los fundamentos de la **programación orientada a objetos**.
 - Genera código **independiente de la arquitectura** de la computadora.



Elementos básicos de JAVA

- Variables, Constantes y Expresiones. Asignaciones
- Tipos elementales. Casting
- Operadores. Precedencia de Operadores
- Estructuras de control
- Estructuras de un programa
- Métodos
- Método Main
- Entrada/Salida

Elementos básicos de JAVA

- Variables, Constantes y Expresiones. Asignaciones
- Tipos elementales. Casting
- Operadores. Precedencia de Operadores
- Estructuras de control
- Estructuras de un programa
- Métodos
- Método Main
- Entrada/Salida

Variables, Constantes y Expresiones

Declaración

[<modificador>] <tipo> <identificador> [, <identificador>] ;

```
int i,I,j101;  
static char fin = '.';  
boolean eureka;
```



La declaración establece el nombre, tipo, alcance y determina si es variable o constante.
Una declaración de constante establece también el valor.

```
final float PI = 3.14f;
```



Tipos elementales

Nombre del Tipo	Valor	Memoria requerida	Rango
byte	integer	1 byte	-128 a 127
short	integer	2 bytes	-32768 a 32767
int	integer	4 bytes	-2147483648 a 2147483647
long	integer	8 bytes	
float	floating-point	4 bytes	
double	floating-point	8 bytes	
char	single character	2 bytes	UNICODE
boolean	true or false	1 bit	

Variables, Constantes y Expresiones

Operadores relacionales

igual	==
distinto	!=
menor	<
menor o igual	<=
mayor	>
mayor o igual	>=

Operadores booleanos

Conjunción (and)	&
Disyunción (or)	
Negación (not)	!

Variables, Constantes y Expresiones

Completa

$a \ \& \ b$

Cortocircuito

$a \ \&\& \ b$

En la evaluación **completa** se evalúan siempre **todos los términos** de la expresión.

En la evaluación en **cortocircuito** sólo se evalúa **lo necesario** para verificar el resultado de la expresión.

Ejemplo: Si a es falso, ya no es necesario evaluar b , porque sin importar su valor la expresión $a\&\&b$ será falsa.



Variables, Constantes y Expresiones

<code>() , []</code>
<code>-num, --, ++, !</code>
<code>*, /, %</code>
<code>+, -</code>
<code>>, <, =, >, =<</code>
<code>==, !=</code>
<code>&&</code>
<code> </code>
<code>=, +=, -=, *=, /=, %=</code>

Variables, Constantes y Expresiones

Asignación

`<ident> = <expresion>;`
`<tipo><ident> = <expresion>;`



```
boolean estado;  
estado = true;  
int a = 3;
```

Cuando la expresión que aparece a la derecha de una asignación no coincide con el tipo de la variable que está a la izquierda puede producirse una **conversión automática o un error.**

Variables, Constantes y Expresiones

La conversión entre tipos es **automática** si:

- Los dos tipos son compatibles
- El tipo de la variable destino es de un rango mayor al tipo de la variable que se va a convertir.

Jerarquía de Tipos

byte → short → int → long → float → double

Variables, Constantes y Expresiones

Cuando el número a convertir está **fuera del rango del tipo** a que se quiere convertir, la conversión automática es imposible.

Este error puede salvarse mediante una **conversión forzada** (explícita), que se denomina **casting**.

La sintaxis para realizar un casting es

destino = (tipo_destino) valor

En el caso de los tipos numéricos, al "recortar" el valor de la variable de rango mayor se **pierde precisión**

Elementos básicos de JAVA

- Variables, Constantes y Expresiones. Asignaciones
- Tipos elementales. Casting
- Operadores. Precedencia de Operadores
- Estructuras de control
- Estructuras de un programa
- Métodos
- Método Main
- Entrada/Salida

Estructuras de Control

Bloques

{ [<instruccion>]* }

```
{      promedio = total / n ;  
      System.out.print("El promedio es ");  
      System.out.println(promedio); }  
{      int temp;  
      temp = x;  
      x = y;  
      y = temp; }
```

Estructuras de Control

Un **bloque** define un **nuevo ambiente de referenciamiento**.

- Las variables declaradas dentro de un bloque son **locales** y no son visibles fuera de él.
- Adoptaremos la convención de **declarar las variables** de un bloque **al principio** e inicializarlas explícitamente.

Estructuras de Control

Condicional

if (<expresion booleana>)

 <instruccion>

else

 <instruccion>

```
if (x > y)
    max = x;
else
    max = y;
```

```
if (x > y) {
    max = x;
    min = y ;
}
else {
    max = y;
    min = x ;
}
```

```
if (x > y) {
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

Estructuras de Control

```
estado = 'B';  
if(promedio > 7)  
    if(inasistencias<3)  
        estado = 'A';  
    else  
        estado = 'C';
```

```
estado = 'B';  
if(promedio >7){  
    if(inasistencias<3)  
        estado = 'A';  
}  
else  
    estado = 'C';
```

Iteración

```
switch (<expresion>) {  
    [ case <constante> : <instruccion>]*  
    default: <instruccion>  
}
```

Estructuras de Control

```
switch ( nota ) {  
    case 10:  
    case 9:  
        estado= 'A' ;  
        break;  
    case 8:  
    case 7:  
        estado= 'B' ;  
        break;  
    case 6:  
    case 5:  
        estado= 'C' ;  
        break;  
    default :  
        estado= 'D' ;  
        break;  
}
```

Estructuras de Control

Iteración

**while (<expresion booleana>)
 <instruccion>**

**do
 <instruccion>
while(<expresion booleana>)**

```
int numero;  
int digitos = 0;  
System.out.println ("Ingrese el  
numero");
```

```
numero = scanner.nextInt();
```

```
while ( número > 0 ) {  
    numero /=10;  
    digitos++;  
}
```

```
System.out.println(digitos);
```

Estructuras de Control

Iteración

for (<asignacion> ; <expresion> ; <expresion>)
 <instruccion>

```
int n;  
for ( n = 1 ; n <= 10 ; n++ )  
    System.out.println( n, n*n );
```

```
for ( int n = 1 ; n <= 10 ; n++ )  
    System.out.println( n, n*n );
```



Estructuras de Control

Iteración



```
int n, int sum;  
for ( n = 1, sum = 0; n<=10 ; sum = sum+n, n++);
```

```
int n, sum;  
for ( n = 1, sum = 0 ; n <= 10 ; n++)  
    sum = sum + n ;
```



```
int sum = 0; int n;  
for ( n = 1 ; n <= 10 ; n++ )  
    sum = sum + n ;
```

```
int sum = 0;  
for ( int n = 1 ; n <= 10 ; n++ )  
    sum = sum + n ;
```



Elementos básicos de JAVA

- Variables, Constantes y Expresiones. Asignaciones
- Tipos elementales. Casting
- Operadores. Precedencia de Operadores
- Estructuras de control
- Estructuras de un programa
- Métodos
- Método Main
- Entrada/Salida

Estructura de un Programa

Símbolos separadores

(){}[];.,

Comentarios

De una sola línea

// ...

De múltiples líneas

/* ... */

```
Principal.java x
1 package com.carlos;
2
3 /**
4  * Esta es la clase principal del programa.
5  * @author carlos
6  * @version 1.0
7  * @since 19/08/2019*/
8
9 public class Principal {
10
11     public enum Dias {Lunes, Martes, Miércoles};
12
13     public static void main(String[] args) {
14
15         int num1 = 9;
16         float num2 = 9.3f;
17         float suma = num1 + num2;
18
19         System.out.println(suma);
20     }
21 }
```

Métodos

[<Modificador>] [<Tipo del Resultado>]
 <Identificador del Bloque> ([<Parámetros Formales>]) {
 <bloque>
 }

```
public int obtenerMayor (int numero1, int numero2) {  
    if (numero1 > numero2) {  
        return numero1; }  
    else {  
        return numero2; }  
}
```

Métodos Main

Para que un programa en JAVA pueda ejecutarse es necesario definir una clase que incluya un método llamado **main**.

```
Class Hello{ //define una clase llamada Hello
```

```
public static void main (String args[]){
```

```
// define el método main. IMPORTANTE no alterar el encabezamiento de main.
```

```
System.out.println( "Hello World!");
```

```
//Se muestra el texto "Hello World!" en pantalla
```

```
}
```

```
}
```



Elementos básicos de JAVA

- Variables, Constantes y Expresiones. Asignaciones
- Tipos elementales. Casting
- Operadores. Precedencia de Operadores
- Estructuras de control
- Estructuras de un programa
- Métodos
- Método Main
- Entrada/Salida

Entrada: Clase Scanner

La **clase Scanner** es parte del paquete *java.util*
Se utiliza para leer datos de entrada, como texto o números.

Para utilizarlo es necesario:

1. Importar la clase
2. Crear un objeto de tipo *Scanner*
3. Leer los datos
4. Cerrar el Scanner

1. Importar la clase

```
import java.util.Scanner;
```

2. Crear un objeto de tipo Scanner

```
Scanner scanner = new Scanner (System.in);
```

3. Leer los datos

```
String nombre = scanner.nextLine();
```

```
int edad = scanner.nextInt();
```

3. Leer los datos

next(): Realiza una lectura hasta que encuentra el primer espacio.

nextBoolean(): Lee y devuelve un valor booleano (*true* o *false*).

nextDouble(): Lee el siguiente número decimal (tipo *double*)

nextFloat(): Lee el siguiente número decimal (tipo *float*)

nextInt(): Lee el siguiente número entero (tipo *int*)

nextLine(): Lee y devuelve una línea completa de entrada, incluyendo los espacios hasta que encuentra un salto de línea.

4. Cerrar el Scanner

```
scanner.close();
```

Salida

System.out.print(...): Imprime en la consola sin añadir un salto de línea al final. Esto significa que la siguiente escritura la hará en la misma línea.

System.out.println(...): Imprime el texto proporcionado y añade un salto de línea al final. La siguiente escritura se hará en una nueva línea.

Ejemplo:

```
System.out.println("Hola Mundo");
```

¿Preguntas?



<https://www.manualweb.net/java/>